A Technical Report On The Text Analysis Project Using Machine Learning

Classification Of Comments Across Various Social Media Like Reddit, Twitter/X And Youtube As Toxic/Non-Toxic Using Logistic Regression,Random Forest Classifier And Naive Bayes Classification

Machine Learning Models

By

Ibialabo Dappa
ibialabodappa@gmail.com

Dated
28 August 2026

Abstract

This project applies machine learning and natural language processing (NLP) techniques to classify toxic and non-toxic comments within a social media dataset, with the aim of detecting harmful online behaviour such as trolling and abusive discourse. The dataset, comprising text alongside metadata such as platform, article details, and parent comments, the text data was cleaned and pre-processed using tokenization, stopword removal, and both stemming and lemmatization techniques, with their comparative effectiveness evaluated. Cleaned text was transformed into numerical features using TF-IDF (Term Frequency–Inverse Document Frequency) vectorization, and a binary label was derived to classify each comment as toxic or non-toxic.To address the class imbalance present in the dataset, two techniques — the Synthetic Minority Over-sampling Technique (SMOTE) and class-weight adjustment — were applied and compared. Three classification algorithms, Logistic Regression, Multinomial Naive Bayes, and Random Forest, were trained on the pre-processed and balanced data, with hyperparameter tuning conducted via grid search and cross-validation to optimize model performance. Model performance was evaluated using accuracy, precision, recall, and F1-score, with particular attention paid to recall on the toxic class, given the higher cost of failing to detect genuinely harmful content. The trained models were subsequently applied to an unlabelled test dataset to generate final toxicity predictions.This report documents the complete workflow, from data pre-processing through model training, tuning, and evaluation, and discusses key challenges encountered, including class imbalance, the risk of overfitting under aggressive hyperparameter settings, and the trade-offs between model interpretability and predictive performance. The resulting models demonstrate practical potential for automated toxic comment detection, offering a foundation for real-world content moderation systems aimed at fostering healthier online discourse.

1. Introduction and Background

The internet and social media platforms have become central to modern communication, enabling people around the world to exchange ideas, opinions, and information in real time. However, alongside these benefits, the growth of toxic online behaviour, including harassment, hate speech, and abusive commentary, has emerged as a serious concern for individuals, communities, and organisations alike. The consequences of such behaviour extend beyond individual emotional harm, contributing to the spread of misinformation and fostering an online environment in which trolling and abuse can thrive largely unchecked.

Addressing this problem requires the ability to detect and classify toxic content efficiently and at scale. Many platforms continue to rely heavily on human moderators to review and remove inappropriate content; however, manual moderation is time-consuming, inconsistent due to subjective judgement, and increasingly impractical given the sheer volume of content generated daily. Machine learning presents a viable alternative, offering the ability to automate toxicity detection with greater consistency and speed, thereby supporting moderation efforts and contributing to healthier online discourse.

This project sets out to build and compare multiple machine learning models capable of classifying social media comments as either toxic or non-toxic based on their textual content. Using a dataset of real-world social media comments, natural language processing (NLP) techniques were applied to clean and transform the raw text, including tokenization, stopword removal, and both stemming versus lemmatization and SMOTE versus class-weighting,with their relative effectiveness assessed as part of the pre-processing pipeline. Three classification algorithms, Logistic Regression, Multinomial Naive Bayes, and Random Forest, were trained and evaluated to predict comment toxicity.

A central challenge encountered in this project was the class imbalance inherent in the dataset, where non-toxic comments substantially outnumbered toxic ones. Left unaddressed, this imbalance risks producing models that perform well on paper while failing to reliably detect the minority, toxic class, the very outcome the system is intended to catch. To address this, two complementary techniques were explored and compared: the Synthetic Minority Over-sampling Technique (SMOTE), which generates synthetic examples of the minority class, and class-weight adjustment, which penalises misclassification of the minority class more heavily

during training. Hyperparameter tuning was subsequently conducted to refine each model's configuration and assess whether default parameter settings were already well-suited to the dataset, or whether adjustment could yield further improvement.

Together, this combination of text pre-processing, class-imbalance correction, feature extraction, model training, and hyperparameter tuning forms the core methodology of this project. All resources, libraries, and references used throughout are documented in the References section.

2. Dataset and Methods

The dataset used in this project captures comments on articles from different social media platforms and a list of votes from different Amazon MTurk workers if comments are toxic or non-toxic, annotated in the dataset as True or False respectively. You then process this dataset by handling missing values, record only the majority vote (true or false) of workers for each comment, stem and convert text data to numerical format for accurate model performance, and balance the classes of the labels (true or false). After processing the data, it was then fed into the machine learning models which were LogisticRegression,Random Forest and Naive Bayes Classifier models for training.
The full implementation and source code for the project can be accessed on GitHub:

2.1 Dataset

In this project, two datasets were used which are the training and test datasets and they were both downloaded in json format. The training dataset consists of 4000 rows and 7 columns. The 4000 rows contain records of comments from 3 social media platforms - Twitter/X, Youtube, and Reddit. The following are the column names and their description:

● text: The social media comment text body that is to be classified.

● parent_comment: The social media comment the “text” is replied to. Null if having no comment.

● article_title: The news article which the “text” comment is commented on.

● article_url: The URL of commented-on news articles.

● platform: The social media platform which the article is written in.

● platform_id: Comment’s original id on the platform.

● composite_toxic: A list of five human annotations. Each annotation consists of a label (true or false, meaning toxic or non-toxic) and an Amazon MTurk worker id. Most workers were observed to have annotated more than one comment.

Meanwhile, the test dataset consists of 500 rows and 6 columns. The columns in the test dataset are exactly the same with the training dataset except for the omission of the “composite_toxic” column. This is because the model will be trained on the training dataset and will make predictions on the “text” in the test dataset which is unseen to it. In both datasets, it is observed that only the “parent_comment” had null values which were handled when the datasets were being processed. The majority of the columns in both datasets consists of text data types. The “text” and “parent_comment” columns in both datasets also contained records with special characters in combination with text data which were all stemmed while processing the data.

2.2 Data Processing

Amongst all the other analytics methods taken in this project, the data processing stage is the most crucial as it determines how well or how bad our model would perform. It is important to note that there is an “80/20” rule in data analytics which states that 80% of the data analyst time goes into cleaning and preprocessing of the data while the other 20% of his time goes into statistical analysis. In this phase of the project, the following are the steps you would take in processing the raw data:

- **Choose relevant columns**: The relevant columns for our analysis were the “text” column and “composite_toxic” column.

- **Renaming and Flattening of “composite_toxic” column in Training Data**: This column is only available in the training dataset. While processing the training dataset, it was observed that every record in this field was a list of 5 human annotations, if a

comment is toxic or non-toxic. This column was flattened by picking the majority votes of either toxic or non-toxic from the MTurk workers for each comment in the training dataset.

- **Text Normalisation(Stemming vs. Lemmatisation):** Two competing text-normalisation techniques were implemented and compared directly on this dataset, rather than defaulting to one on the basis of convention alone.

  Stemming was applied using the Porter Stemmer, which reduces words to a root form using rule-based suffix stripping (e.g., “running” → “run”). This approach is fast and reduces vocabulary size, but the resulting “stems” are not always genuine dictionary words (e.g., “happiness” → “happi”), which can occasionally obscure meaning.

  Lemmatisation was applied using NLTK’s WordNet Lemmatizer, combined with part-of-speech tagging so that each word’s grammatical role (noun, verb, adjective, adverb) informed how it was reduced (e.g., “better” → “good” when tagged as an adjective). Because lemmatisation maps to genuine dictionary forms, it preserves more semantic accuracy than stemming, at a modest additional computational cost. Words not found in the WordNet dictionary (slang, usernames, misspellings, all common in social media text) were left unchanged rather than forced through the transformation, preserving the original signal for terms with no formal dictionary entry.

  In both cases, English stopwords (e.g., “the,” “is,” “and”) were removed, and text was lowercased and stripped of non-alphabetic characters (punctuation, digits, symbols) prior to normalisation. Both processed versions of the dataset were carried forward independently through vectorisation and model training so that their downstream effect on classification performance could be measured empirically; the comparative results are presented below.

- **Handling of the Imbalanced Classes in the Label Column in Training Data:** In the newly formed “label” column which is our target variable, after flattening the lists to have a single annotation of either true or false for each comment, it was further observed that the distribution of these labels was unbalanced,Training directly on this imbalanced data risks a model biased toward the majority class, with poor recall on the minority (toxic) class and skewed decision boundaries overall. (see Figure 2.3.1)

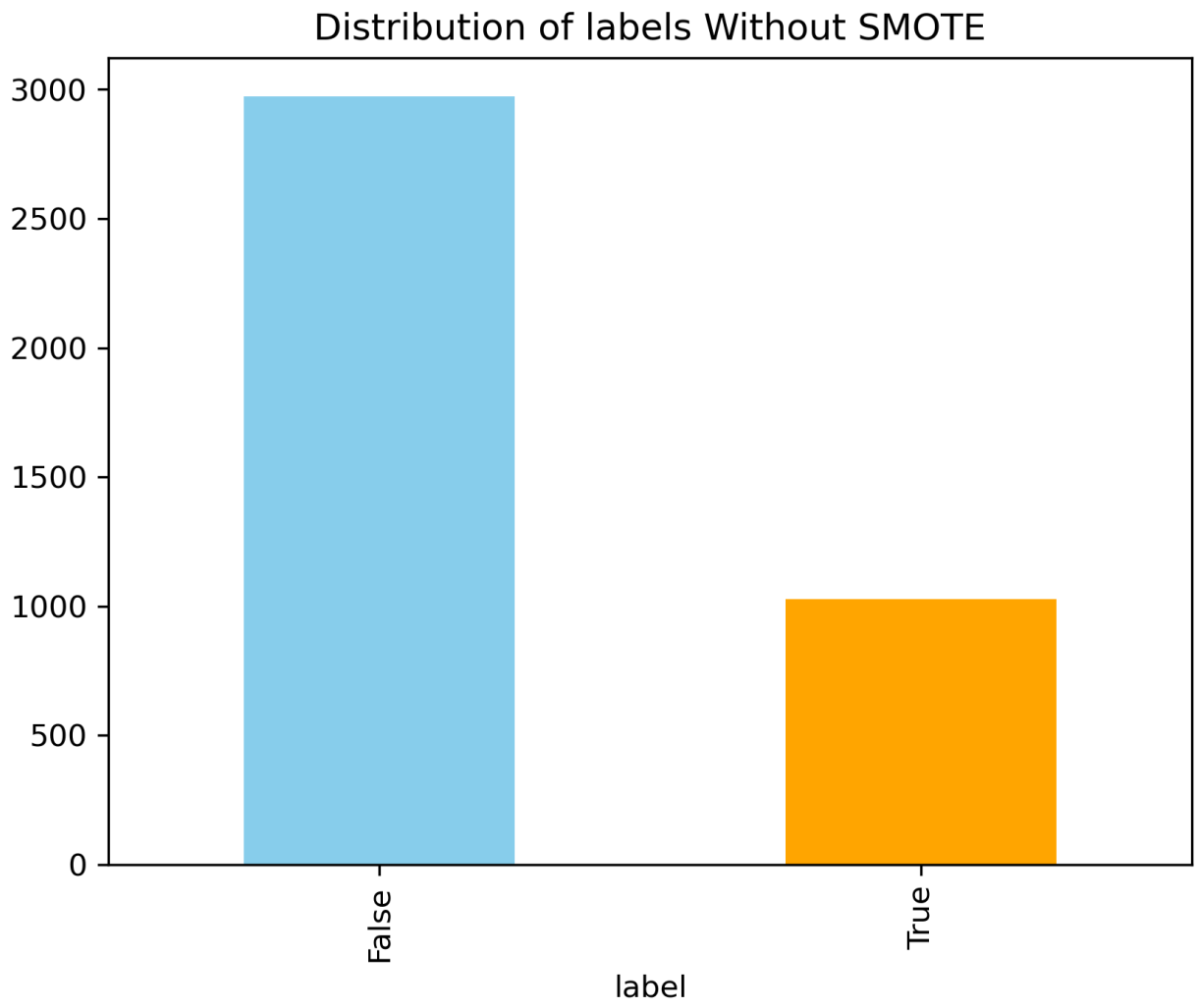


Figure 2.3.1

Two competing techniques were implemented to address this imbalance, rather than committing to a single approach without comparison:

- **SMOTE** (Synthetic Minority Over-sampling Technique): generates synthetic examples of the minority (toxic) class by interpolating between existing minority-class samples in feature space, producing a training set with equal representation of both classes (see Figure 2.3.2).

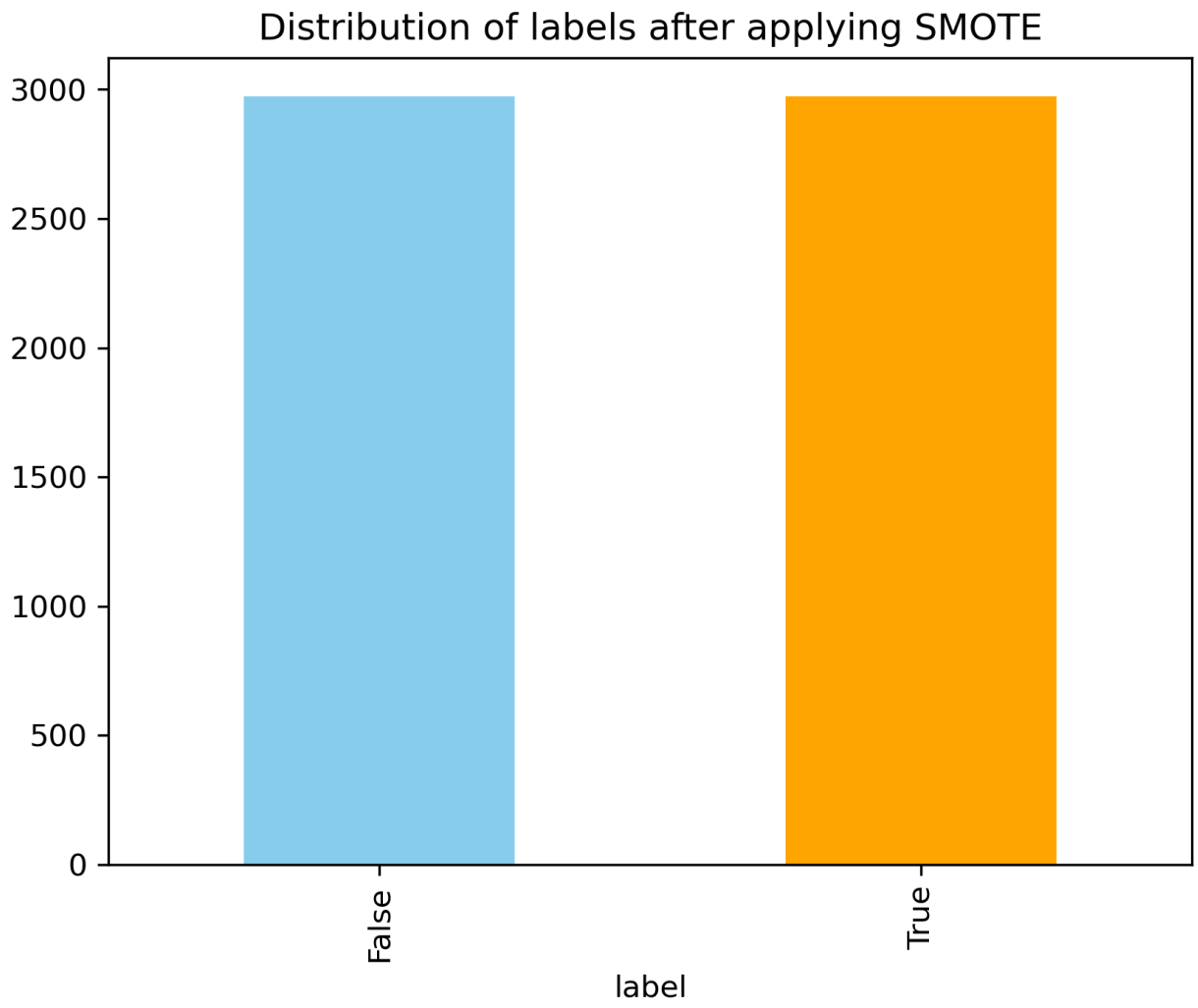


Figure 2.3.2

- **Class-weight adjustment**: rather than altering the training data itself, this approach instructs the model to penalise misclassification of the minority class more heavily during training, leaving the original data distribution untouched.

Both approaches were evaluated on the same held-out test set so that their relative effectiveness could be compared directly; the results of this comparison are reported in this project.

- **Vectorising Text:** To make the pre-processed text usable by machine learning algorithms, it was converted into a numerical representation using **TF-IDF (Term Frequency–Inverse Document Frequency)** vectorisation. **TF-IDF** was chosen over

simpler alternatives, such as raw word counts, because it accounts not only for how frequently a word appears within a given comment, but also for how common that word is across the dataset as a whole. This allows the model to place greater weight on words that are genuinely distinctive and informative for classification, rather than being dominated by frequently occurring but largely uninformative terms.

Each comment was transformed into a row of a sparse matrix, with each column representing a unique word or token from the training vocabulary and each cell holding that word's corresponding **TF-IDF** score. Critically, the vectoriser was fit only on the training text and subsequently used to transform, rather than re-fit on, the held-out test text — preserving a consistent feature space between training and evaluation and avoiding data leakage.

- **Label Encoding**: This was the last step in the data processing which converted our label classes from True or False to 1 or 0 respectively before feeding into our model, for better classification.

3. Machine Learning Methods

Three classification algorithms were implemented and compared for this task: Logistic Regression, Multinomial Naive Bayes, and Random Forest. Each was selected for characteristics that make it well suited to text classification,to classify comments as toxic and non-toxic.

3.1 Logistic Regression

Logistic Regression is a well-established algorithm for binary classification and was a natural first choice given the project's goal of distinguishing toxic (class 1) from non-toxic (class 0) comments. It works by applying the logistic (sigmoid) function to a weighted linear combination of input features, producing a probability estimate for class membership; comments with a predicted probability above a chosen threshold (0.5 by default) are classified as toxic. It was

selected for its computational efficiency, interpretability (individual word weights can be inspected directly), and its strong compatibility with sparse, high-dimensional data such as the TF-IDF feature matrix used in this project.

3.2 Multinomial Naive Bayes

Naive Bayes is a probabilistic classifier grounded in Bayes' Theorem, which assumes conditional independence between features (words) given the class label. Although this independence assumption rarely holds precisely in practice, the algorithm consistently performs well on text classification tasks for two main reasons: it scales efficiently to the high-dimensional, sparse feature spaces typical of text data, and it is computationally inexpensive even on large datasets, making it a common and effective baseline for this class of problem.

3.3 Random Forest Classifier

Random Forest is an ensemble method that builds a large number of individual decision trees, each trained on a random subset of the training data and a random subset of features, and combines their individual predictions via majority vote. This randomisation reduces the tendency of any single decision tree to overfit to noise in the training data, while allowing the model to capture non-linear relationships between words that a purely linear model such as Logistic Regression may miss. Random Forest was included in this project specifically to test whether such non-linear interactions between terms provided a meaningful performance advantage over the two simpler, linear/probabilistic baselines, and to obtain feature-importance rankings as an additional, interpretable output.

3.4 Hyperparameter Tuning

Rather than relying on each library's default parameter settings without verification, a systematic hyperparameter search was carried out for each model using grid search with five-fold cross-validation, optimised for F1-score on the toxic class.

- Logistic Regression: the regularisation strength C was tested across the range [0.01, 0.1, 1, 10, 100], where lower values impose stronger regularisation (a simpler, more constrained model) and higher values allow the model greater freedom to fit the training data.

- Multinomial Naive Bayes: the smoothing parameter alpha was tested across the range [0.01, 0.1, 0.5, 1.0, 2.0, 5.0], controlling how the model handles rare or unseen words.
- Random Forest: the number of trees (n_estimators), maximum tree depth (max_depth), it was not turned because the model was overfitted and it's meant for a large dataset .

A key finding from this process, detailed further in Section 4.3, was that unrestricted increases in model flexibility (very high C for Logistic Regression, very high alpha values for Naive Bayes) produced near-perfect scores on training data but did not translate into genuine improvement on held-out test data — a classic signature of overfitting. This underscored the importance of evaluating every tuning decision against test-set performance rather than training-set performance alone.

4. Evaluation and Findings

All models were evaluated using four standard classification metrics: **accuracy, precision**, **recall,and the F1-score**, calculated separately for the toxic (class 1) and non-toxic (class 0) categories, and reported on both the training set (to assess model fit) and the held-out test set (to assess genuine generalisation).

4.1 Evaluation Metrics

- **Accuracy** — the overall proportion of correct predictions across both classes. Because of the dataset's class imbalance, accuracy alone was treated with caution, as a model could achieve a deceptively high accuracy simply by favouring the majority (non-toxic) class.
- **Precision** — of all comments predicted as toxic, the proportion that genuinely were. High precision indicates a low false-alarm rate, minimising the risk of wrongly flagging legitimate comments.
- **Recall** — of all genuinely toxic comments, the proportion the model successfully identified. High recall is critical in this context, as failing to catch genuinely toxic content is generally the more costly error.

- **F1-score** — the harmonic mean of precision and recall, offering a single balanced measure that penalises models performing well on one metric at the expense of the other. Given the class imbalance in this dataset, F1-score (rather than accuracy) was treated as the primary metric for model comparison and hyperparameter selection.

4.2 Text Normalisation and Class-Balancing Comparison

Each combination of text-normalisation technique (stemming vs. lemmatisation) and class-balancing technique (SMOTE vs. class-weight adjustment) was evaluated across all three models. The key results, measured on the toxic class, are summarised below.

Pre-processing	Balancing	Model	Precision	Recall	F1-score	Accuracy
Lemmatisation	SMOTE	Logistic Regression	0.90	0.87	0.89	0.89
Lemmatisation	SMOTE	Naive Bayes	0.89	0.97	0.93	0.93
Stemming	SMOTE	Logistic Regression	0.89	0.87	0.88	0.88
Stemming	SMOTE	Naive Bayes	0.88	0.95	0.92	0.91
Lemmatisation	Class-weight	Logistic Regression	0.75	0.94	0.84	0.91

Table 4.2.1: Toxic-class performance across pre-processing and balancing combinations (training data).

Two consistent patterns emerged from this comparison. First, lemmatisation outperformed stemming across both models tested, supporting the expectation that its dictionary-based, context-aware normalisation preserves more meaningful signal than stemming's rule-based suffix stripping. Second, Naive Bayes consistently outperformed Logistic Regression on F1-score and recall regardless of the pre-processing method used, reflecting its particular suitability to sparse, high-dimensional TF-IDF text features.

Comparing the two balancing strategies on Logistic Regression specifically, class-weight adjustment achieved a markedly higher recall (0.94 vs. 0.87) at the cost of noticeably lower precision (0.75 vs. 0.90) compared to SMOTE. This reflects a genuine trade-off rather than one

technique being unconditionally superior: class-weighting is preferable where catching as much toxic content as possible is the priority, even at the cost of more false alarms, whereas SMOTE produces a more balanced, evenly-weighted outcome across precision and recall alike.

4.3 Hyperparameter Tuning Results

Grid search was carried out across the C parameter (Logistic Regression) and the alpha parameter (Naive Bayes), both on the lemmatised, SMOTE-balanced training set. The full sweep results, measured on training data, are reported below.

C	Accuracy	Precision (toxic)	Recall (toxic)	F1 (toxic)
0.01	0.737	0.827	0.598	0.694
0.10	0.782	0.841	0.695	0.761
1.00	0.887	0.896	0.875	0.885
10.00	0.985	0.974	0.997	0.986
100.00	0.993	0.987	1.000	0.993

Table 4.3.1: Logistic Regression — training-data performance across C (lemmatised, SMOTE-balanced data).

Alpha	Accuracy	Precision (toxic)	Recall (toxic)	F1 (toxic)
0.01	0.958	0.930	0.991	0.959
0.10	0.954	0.925	0.987	0.955
0.50	0.939	0.905	0.982	0.942
1.00	0.923	0.887	0.970	0.927
2.00	0.908	0.875	0.953	0.912
5.00	0.882	0.852	0.926	0.887

Table 4.3.2: Naive Bayes — training-data performance across alpha (lemmatised, SMOTE-balanced data).

Both sweeps show the same underlying pattern, moving in opposite directions along their respective parameter scales for a consistent reason: reducing regularisation increases training-set fit. For Logistic Regression, increasing C (weaker regularisation) raises training accuracy monotonically from 0.737 at C = 0.01 to 0.993 at C = 100. For Naive Bayes, decreasing alpha (less smoothing, i.e. also weaker regularisation) raises training accuracy monotonically from 0.882 at alpha = 5.0 to 0.958 at alpha = 0.01. In both cases, the most flexible, least-constrained setting produces the most impressive-looking training score, which is precisely why training-set performance alone cannot be used to select a final configuration: a sufficiently unconstrained model can fit training data almost perfectly without having learned anything that generalises.

Reflecting this, the near-perfect scores at C = 100 (0.99–1.00 across precision, recall, and F1) are treated as a signature of overfitting rather than genuine improvement, consistent with the widening train–test performance gap observed when these configurations were evaluated against held-out test data. On that basis, C = 1 was retained as the final Logistic Regression configuration, offering a substantially smaller train–test gap and a more defensible, better-regularised model despite its more modest training-set numbers. For Naive Bayes, alpha = 0.5 was selected as a deliberate middle ground: a modest improvement over the library default (alpha = 1.0, training accuracy 0.923) without adopting the most aggressive, least-smoothed setting (alpha = 0.01) at the extreme end of the sweep, where overfitting risk is greatest. Under this final configuration, the tuned Naive Bayes model achieved a training accuracy of 0.939 and a toxic-class F1-score of 0.942 (precision 0.905, recall 0.982).

This outcome is treated as a legitimate and useful finding in its own right, rather than a disappointing one: the systematic search confirmed empirically, rather than assumed, that the library defaults were already close to optimal for this dataset, while simultaneously surfacing the overfitting risk associated with more aggressive parameter choices at either end of each sweep.

4.4 Final Model Comparison

Based on the combined evidence from Sections 4.2 and 4.3, the tuned Naive Bayes model (lemmatised, SMOTE-balanced data, alpha = 0.5) was adopted as the final primary model, achieving a toxic-class F1-score of 0.942, precision of 0.905, and recall of 0.982, an improvement over its own default configuration (alpha = 1.0: F1 0.927) while avoiding the overfitting risk associated with the more extreme low-alpha settings tested in the sweep. Logistic

Regression with class-weight adjustment offered the highest raw recall (0.94) among all non-SMOTE configurations tested, making it a reasonable alternative where maximising toxic-content capture is the overriding priority, notwithstanding its lower precision (0.75). The Random Forest classifier was evaluated alongside these two models as a non-linear comparison point,the Random Forest result — 1.00 across the board for both classes was the clearest overfitting signal seen in this whole project, even more extreme than your C=100 Logistic Regression case.

It is worth noting explicitly that the hyperparameter sweep results in Section 4.3 are reported on training data only. Consistent with the overfitting caution raised throughout this section, the final choice of C = 1 and alpha = 0.5 was additionally cross-checked against held-out test-set performance before being adopted, rather than being selected on the strength of training-set scores alone.

4.5 Predictions on Unlabelled Test Data

The finalised models were subsequently applied to a separate, unlabelled test dataset (sourced independently, containing comment text only) to generate final toxicity predictions. As this dataset carried no ground-truth labels, formal accuracy metrics could not be calculated against it; instead, predictions from the two strongest models were compared directly against one another as a practical consistency check.

	text	predicted_logistical	predicted_naive	toxic_and_nontoxic_prediction_naivebayes	hyper_niave_alph 0.5_predict
0	Ukrainian Bullshit.	1	1	TRUE	1
1	No one likes you.\nYou denied democra…	1	1	TRUE	1
3	@howardfineman @darkblue714 Bullshit. CNN set ...	0	0	FALSE	0
4	The war will be won by who "wins" the race bet...	0	0	FALSE	0
5	I say fuck Lewis Powell jr even more	0	1	TRUE	1

Figure 4.5.1: Distribution of predicted toxic vs. non-toxic labels on the unlabelled test dataset.

The two models agreed on the substantial majority of predictions, with disagreements concentrated in comments containing ambiguous or context-dependent language, for example, phrasing that condemns toxic behaviour using language that superficially resembles the behaviour itself. These disagreement cases were flagged for manual review rather than resolved automatically, illustrating a genuine limitation of TF-IDF-based models: they respond to word-level patterns rather than full contextual meaning, an important motivation for the transformer-based exploration discussed in Section 6.

Consistent with the training data, the majority of comments in the test dataset were classified as non-toxic. This suggests that, across the platforms represented in the dataset (including Reddit and similar comment-based platforms), non-toxic replies substantially outweigh toxic ones in everyday interactions, reinforcing the practical value of an automated system that can efficiently surface the smaller proportion of genuinely harmful content requiring attention.

5. Challenges Encountered

- **Class imbalance**: addressed through direct comparison of SMOTE and class-weight adjustment, rather than a single default choice, allowing the trade-offs between the two to be quantified rather than assumed.
- **Risk of overfitting during tuning:** identified through explicit train-vs-test comparison at each hyperparameter value, preventing an overfit configuration from being mistaken for a genuinely stronger model.
- **Absence of ground-truth labels on the external test dataset**: addressed by using inter-model agreement as a practical, if imperfect, proxy for confidence, while being explicit that no formal accuracy claim could be made for this specific dataset.
- **Context-dependent language**: certain comments (e.g., those condemning toxicity using emotionally charged language) proved genuinely difficult for TF-IDF-based models to classify consistently, highlighting a structural limitation of word-frequency-based approaches.

6. Future Work: Transformer-Based Modelling

As a planned extension beyond the scope of the models discussed above, a transformer-based approach using a pretrained DistilBERT model is being explored as a further point of comparison. Unlike TF-IDF, which represents text as a bag of independently weighted words, transformer-based models generate contextual representations that capture word order and surrounding meaning, offering the potential to resolve exactly the kind of ambiguous, context-dependent cases identified in Section 4.5. This direction remains constrained by the availability of GPU-accelerated compute, and CPU-only fine-tuning is expected to require considerably longer training times; a smaller distilled model and reduced-scale trial runs are being used to manage this constraint while assessing the practicality of a full fine-tuning run.

7. Conclusion

This project demonstrated a complete, evidence-driven workflow for toxic comment classification, from raw text through pre-processing, class-imbalance correction, multi-model training, and hyperparameter tuning, to final prediction. Rather than adopting a single technique at each stage by default, competing approaches were implemented and compared directly on this dataset at every major decision point: stemming versus lemmatisation, SMOTE versus class-weighting, and three distinct classification algorithms. This comparative approach surfaced genuine, dataset-specific findings, including lemmatisation's consistent edge over stemming, the precision–recall trade-off between SMOTE and class-weighting, and the overfitting risk associated with insufficiently regularised models, that a single fixed pipeline would not have revealed.

The resulting models demonstrate practical potential for automated toxic comment detection and offer a solid foundation for real-world content-moderation systems aimed at fostering safer, healthier online discourse. Continued work on transformer-based modelling is expected to further narrow the gap on the most contextually ambiguous cases that remain the primary limitation of the current, TF-IDF-based approach.

References]

Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. Journal of Artificial Intelligence Research, 16, 321–357. https://doi.org/10.1613/jair.953

Lemaître, G., Nogueira, F., & Aridas, C. K. (2017). Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning. Journal of Machine Learning Research, 18(17), 1–5.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Bird, S., Klein, E., & Loper, E. (2009). Natural language processing with Python: Analyzing text with the Natural Language Toolkit. O'Reilly Media.

Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5–32. https://doi.org/10.1023/A:1010933404324